Smart Search — Technical Report

VS Code Extension | Master's Senior Research Project Author: Nawfal Jalloul **Date:** April 2026

---

1. What is Smart Search?

Smart Search is a VS Code extension that brings **semantic code search** to any codebase. Unlike traditional search tools (grep, VS Code's built-in search) that match exact words or patterns, Smart Search understands the *meaning* of code.

Example:

- You type: `"find authentication logic"`
- Traditional search: finds files containing the word "authentication"
- Smart Search: finds `verify_token()`, `check_password()`, `validate_session()` — even if the word "authentication" never appears in those function names

This is the research contribution: **semantic search over code using embeddings and LLMs**, compared against lexical/keyword search.

---

2. High-Level Architecture

The system has three parts:

```
Developer's Machine                Cloud
────────────────────               ──────────────────────────

VS Code Extension (TypeScript)  →  Voyage AI (embedding)
Python Backend (localhost:8000) →  Pinecone (vector storage)
.smart-search/ (local hashes)   →  OpenAI GPT (summaries + line locator)
```

VS Code Extension handles everything local:

- Detecting which files/functions changed
- Splitting code into functions (chunking)
- Running the search UI

Python Backend handles the AI operations:

- Generating plain English summaries via GPT-4o-mini (index time)
- Embedding code + summary via Voyage AI (index time)
- Saving/deleting vectors in Pinecone (index time)
- Embedding search queries via Voyage AI (search time)
- Querying Pinecone for similar functions (search time)
- Asking GPT which specific line matches the query (search time)

Why split? VS Code extensions run in Node.js (TypeScript/JavaScript). The AI APIs (Voyage AI, Pinecone, OpenAI) have mature Python SDKs. Splitting keeps each part in its strongest language.

---

3. Where Data is Stored

Data	Location	Why

Function content hashes	`project/.smart-search/index.json`	Moves with the project, gitignored
Project unique ID	`project/.smart-search/project-id`	Stable UUID, survives folder moves
User unique ID	MD5 of `git config --global user.email`	Same across all machines for same developer
Embeddings (vectors)	Pinecone cloud	1536 floats per function — too large for local
Function summaries	Pinecone metadata (up to 300 chars)	Generated once, retrieved with search results

Why hashes in the project folder?

Previously considered: VS Code global storage (`~/Library/Application Support/Code/...`). Problem: if the user reinstalls VS Code or moves the project folder, hashes are lost and everything re-embeds from scratch (costs money).

By storing hashes in `.smart-search/` inside the project:

- Moving the folder → hashes move with it → nothing re-embeds
- VS Code reinstall → hashes still in the project folder → nothing re-embeds
- Git push → `.smart-search/` is gitignored → hashes don't pollute the repo

---

4. User Identity System

The Problem

If two developers share the same project, they must not overwrite each other's Pinecone vectors.

The Solution

Pinecone namespace = `{projectId}::{userId}`

- **projectId** — UUID stored in `.smart-search/project-id`. Generated once per project. Travels with the project folder.
- **userId** — MD5 hash of the developer's global git email (`~/.gitconfig`). Same across all their machines. Different from teammates.

```
Nawfal's namespace:   a3f2c1d4 :: b7f2a1c5
Ahmed's namespace:    a3f2c1d4 :: c9d8e7f6   ← same project, different user
```

Why git email?

- Already configured on virtually every developer's machine
- Consistent across home PC, work laptop, new computer
- Survives VS Code reinstall (lives in `~/.gitconfig` , not in VS Code)
- Different from teammates by definition

If git email not configured

Extension shows a clear VS Code error notification:

> *"Smart Search needs your git email. Run: `git config --global user.email you@example.com` — then reload VS Code."*

Indexing stops. No silent fallback that would cause inconsistent behavior.

---

5. Indexing Pipeline — How Functions Get Embedded

Two-Level Hashing Strategy

Embedding costs money (Voyage AI API). The goal is to **only embed what actually changed**.

Level 1 — File Hash (Fast Pre-Filter)

1. MD5-hash the entire file content
2. Compare with stored hash in `index.json`
3. If match → file unchanged → **skip everything** (0 API calls)
4. If different → proceed to Level 2

Cost: one MD5 computation per file. Negligible.

Level 2 — Function Hash (Precise Detection)

1. Split the file into individual functions (done locally in TypeScript — no network call)
2. For each function: normalize content (strip blank lines, trailing whitespace) → MD5 hash
3. Compare each function hash with stored hash:
   - **Same hash** → function unchanged → keep existing Pinecone vector
   - **Different hash** → function changed → add to re-embed queue
   - **Not in index** → new function → add to embed queue
   - **In index but missing** → function deleted → add ID to delete queue

Why normalize before hashing?

If a developer adds a blank line or reformats indentation, the bytes change but the logic doesn't. Without normalization, this triggers a re-embed — wasting money on no real change.

Normalization strips:

- Trailing whitespace on each line
- Blank/whitespace-only lines

After normalization, `verify_token()` with an extra blank line has the **same hash** as without it.

LLM Summarization (Before Embedding)

Before embedding a changed function, we ask GPT to describe it in plain English:

```
Input code:  public function getProductInfo($id) {
               $sql = "SELECT * FROM products WHERE id = ?";
               $result = $db->query($sql, [$id]);
               return $result->fetch_assoc();
             }

GPT says: "Fetches a single product's details from the database by its ID.
              Executes a parameterized SQL SELECT query and returns the row."
```

This summary is prepended to the code before embedding:

```
Function: getProductInfo
Fetches a single product's details from the database by its ID.
public function getProductInfo($id) { ... }
```

Why this dramatically improves scores: Without the summary, the vector only captures syntax. With it, the vector also captures English meaning — so "fetch product from database by id" now matches at 80%+ instead of 45%.

The summary is also stored in Pinecone metadata (≤300 chars) and displayed in the search results UI.

Batched Embedding

Instead of one Voyage AI API call per function, all changed functions from one file are sent in a **single batch call**. Voyage AI supports large batch sizes. This is faster and uses fewer API rate-limit tokens.

The Full Flow (First Run)

```
VS Code opens project
  → Check git email (error if missing)
  → Read or create project-id
  → Build namespace = projectId :: userId
  → Load index.json (empty on first run)
  → Walk all workspace files
  → For each file:
      → Hash file → compare with index
      → If changed: chunk into functions (local, TypeScript)
      → For each changed function:
          → Normalize + hash → compare with index
          → If changed: add to embed queue
      → For each function in embed queue:
          → Summarize via GPT-4o-mini (plain English description)
      → Send ALL functions in one Voyage AI batch call (summary + code → vector)
      → Save new vectors + summaries to Pinecone (under namespace)
      → Save new hashes to index.json
  → Remove deleted files from index + Pinecone
  → Show "X files updated" in status bar
```

On Every File Save

Same flow, but only for the one saved file. If the file hash matches — nothing happens at all.

---

6. Code Chunker — Splitting Code into Functions

Chunking happens entirely in TypeScript inside the extension. No network call, no Python involved.

Supported Languages

Language	Strategy
Python	Indentation-based (`def`, `async def`, `class`)
JavaScript / TypeScript / JSX / TSX	Brace-counting + arrow functions

Java	Brace-counting
Go	Brace-counting
Rust	Brace-counting
C / C++	Brace-counting
C#	Brace-counting
PHP	Brace-counting
Ruby	`end` keyword counting
Swift	Brace-counting
Kotlin	Brace-counting
Other	Whole file as one chunk (fallback)

Chunk Format

Each chunk has:

- **id** — unique ID: `"src/auth.py::verify_token"` (relative path, portable)
- **name** — function name
- **type** — `function`, `method`, or `file` (class chunks are excluded — individual methods already cover the same code and class chunks always outscore their own methods in search)
- **content** — full source code (max 6000 characters)
- **start_line** / **end_line** — position in file
- **language** — programming language

Why relative paths in chunk IDs?

If chunk IDs used absolute paths (`/Users/nawfal/projects/myapp/src/auth.py::verify_token`), moving the project folder would make all existing Pinecone vectors unreachable — the IDs would no longer match.

Using relative paths (`src/auth.py::verify_token`) means the ID is the same regardless of where the project lives on disk.

Why class chunks are excluded

When a PHP/Java/JS class is chunked, the chunker produces two levels:

- The **class** chunk (entire class body = all methods combined)
- Each **method** as its own separate chunk

Because the class chunk contains all methods' code, it always scores higher than individual methods in Pinecone — polluting results. Since individual methods are already indexed, class-level chunks add no value and are filtered out before embedding.

---

7. Pinecone Vector Storage

Each function is stored in Pinecone as:

```
ID:       "src/auth.py::verify_token"
Vector:   [0.21, -0.54, 0.87, ...] (1536 floats)
Metadata: {
  file:       "src/auth.py"
  name:       "verify_token"
  type:       "function"
  start_line: 5
  end_line:   25
  content:    "def verify_token(token):\n    ..." (first 1000 chars)
  summary:    "Validates a JWT token and returns True if valid, False otherwise."
}
Namespace: "a3f2c1d4::b7f2a1c5"   ← projectId :: userId
```

Metadata field limits:

- `content` : first 1000 chars (Pinecone metadata size limit)
- `summary` : first 300 chars (GPT-generated description)

Full source is always on disk — file + line numbers get you there.

Upsert vs Insert

Pinecone uses "upsert" — if a vector with this ID already exists, it's replaced. If not, it's created. This means when a function changes:

1. Delete old vector by ID
2. Upsert new vector with same ID (new embedding + new summary)

---

8. Search

Normal Search

- Query sent to Python `/search` endpoint
- Python walks the entire workspace with regex matching
- Supports: Match Case, Match Whole Word, Use Regex, Include/Exclude glob filters
- Returns: file path, line number, matched content, character-level match positions
- Results rendered in VS Code webview
- Clicking a result opens the file at the exact line with text highlighted

Replace

- Single replace: uses VS Code `WorkspaceEdit` API — supports Ctrl+Z undo
- Replace All: replaces from bottom-to-top across all files so character positions stay accurate

AI Search — Full Pipeline

1. **Query embedding** — User's natural language query is embedded via Voyage AI (`input_type="query"`) into a 1536-float vector
2. **Pinecone query** — That vector is compared against all stored function vectors in the user's namespace (cosine similarity). Top 10 matches returned.
3. **Threshold filter** — Results below the configured minimum score are removed (default 35%, user can set 1–100 in the UI)
4. **Sort** — Remaining results sorted by score descending (most relevant first)

5. **LLM line locator** — For the top 5 results, GPT reads the matched function's code (from disk) and identifies which specific line best answers the query. Returns `{line: 21, content: "..."}`
6. **Display** — Each result card shows:
   - Function name, type badge, score percentage
   - Plain English summary (generated at index time, stored in Pinecone)
   - Green arrow `→ line 21` pointing to the specific matched line
   - Clicking opens the file directly at that line

Why the two-stage LLM approach:

- At **index time**: GPT generates summaries → better embedding quality → higher semantic scores
- At **search time**: GPT locates the exact line → user doesn't have to read the whole function

---

9. Key Design Decisions

Decision	Chosen Approach	Why
Who chunks the code?	TypeScript extension	Removes one network call per file
Where are hashes stored?	`.smart-search/` in project	Survives folder moves and VS Code reinstall
How is user identified?	MD5(git global email)	Same across all machines; survives reinstall
How are projects isolated in Pinecone?	Namespace = projectId::userId	Each user's vectors never mix
How are chunk IDs formed?	Relative file path :: function name	Portable across folder moves
How to avoid re-embedding on format changes?	Normalize before hashing	Saves API cost
How to reduce embedding API calls?	Batch all changed functions in one call	Faster + cheaper
Which embedding model?	`voyage-code-2` (Voyage AI)	Trained on code; asymmetric search (document vs query modes); higher scores than general-purpose models
How to improve semantic scores?	LLM summary prepended before embedding	English meaning in the vector; "fetch product by ID" matches `getProductInfo` at 80%+
How to show exact relevant line?	LLM line locator at search time	Pinecone returns function-level matches; GPT narrows to one line
Classes in search results?	Excluded at index time	Class chunks always outscore their own methods; individual methods are already indexed
What if git email not configured?	Show error, stop indexing	No silent bad behavior

---

10. Research Comparison: Old vs New Search

Aspect	Traditional (grep/regex)	Smart Search v1 (embeddings only)	Smart Search v2 (embeddings + LLM)
Query type	Exact text / pattern	Natural language / concept	Natural language / concept
Finds renamed functions?	No	Yes	Yes
Finds similar logic with different names?	No	Yes	Yes
Semantic score quality	N/A	40–55% (raw code embedding)	75–90% (code + LLM summary)
Result precision	Exact line	Function-level (lines 5–42)	Specific line (→ line 21)
Requires understanding code?	No	Yes (via embeddings)	Yes (via embeddings + LLM)
Cost	Free	Voyage AI ($0.00012/1K tokens)	Voyage AI + GPT-4o-mini (~$0.001/search)
Speed	Very fast (local)	~500ms (embed query + Pinecone)	~1–2s (embed + Pinecone + GPT)
Setup	None	API keys + Python backend	API keys + Python backend

---

11. Running the Project

Prerequisites

- Node.js + npm
- Python 3.8+
- git globally configured email
- Voyage AI API key (free tier: 200M tokens/month — requires payment method on file)
- Pinecone API key + index (1536 dimensions, cosine metric)
- OpenAI API key (GPT-4o-mini for summaries and line locator — same key as embeddings)

Setup

```
# 1. Install Python dependencies
cd backend
pip install -r requirements.txt

# 2. Configure API keys in backend/.env
OPENAI_API_KEY=sk-...          # used for embeddings (voyage) + GPT summaries/line
locator
```

```
VOYAGE_API_KEY=pa-...
PINECONE_API_KEY=pcsk_...
PINECONE_HOST=https://your-index.svc.pinecone.io

# 3. Start Python backend
python3 server.py

# 4. Compile TypeScript extension
cd ..
npm install
npx tsc

# 5. Press F5 in VS Code to launch extension
```

What happens after F5

1. A new VS Code window opens (Extension Development Host)
2. Status bar shows: `⟳ Smart Search: indexing 1/47...`
3. For each changed function: GPT generates a summary, Voyage AI embeds it
4. Status bar shows: `✓ Smart Search: 47 files updated`
5. Open Smart Search from Command Palette → search works

---

12. File Structure

```
smart-search/
├── src/                          ← TypeScript extension source
│   ├── extension.ts              ← Entry point, activation, event listeners
│   ├── indexer/
│   │   ├── workspaceIndexer.ts   ← Core indexing logic, two-level hashing
│   │   ├── chunker.ts            ← Code splitter (12+ languages, local)
│   │   ├── localIndex.ts         ← Read/write .smart-search/index.json
│   │   ├── projectId.ts          ← Generate/read project UUID
│   │   └── userId.ts             ← Get user ID from git email
│   ├── handlers/
│   │   ├── searchHandler.ts      ← Route search queries to Python
│   │   └── replaceHandler.ts     ← Apply replacements in files
│   └── utils/
│       └── webviewManager.ts     ← Load frontend HTML into VS Code panel
│
├── backend/                      ← Python server
│   ├── server.py                 ← HTTP server (localhost:8000)
│   ├── summarizer.py             ← GPT-4o-mini: generate plain English function
summaries
│   ├── line_locator.py           ← GPT-4o-mini: find the exact line matching a query
│   ├── embedder.py               ← Voyage AI embedding (batched, asymmetric modes)
│   ├── pinecone_client.py        ← Pinecone upsert/delete/query (with namespace)
│   ├── search.py                 ← Regex/text file search (normal mode)
│   ├── config.py                 ← Default threshold and ignored folders
│   ├── requirements.txt          ← Python dependencies
│   └── .env                      ← API keys (never committed)
```

```
│
├── frontend/                    ← Search UI (inlined into VS Code webview)
│   ├── index.html               ← UI structure
│   ├── styles.css               ← Styling
│   └── main.js                  ← UI logic, VS Code message passing
│
├── reports/                     ← This folder
│   ├── documents/               ← This document
│   └── workflows/               ← Mermaid diagrams (run at mermaid.live)
│
├── out/                         ← Compiled TypeScript (generated, don't edit)
├── package.json                 ← Extension metadata + build scripts
└── tsconfig.json                ← TypeScript compiler config
```